PROGRAMMING FUNDAMENTALS

CS – 101

Instructor: Maria N. Samad

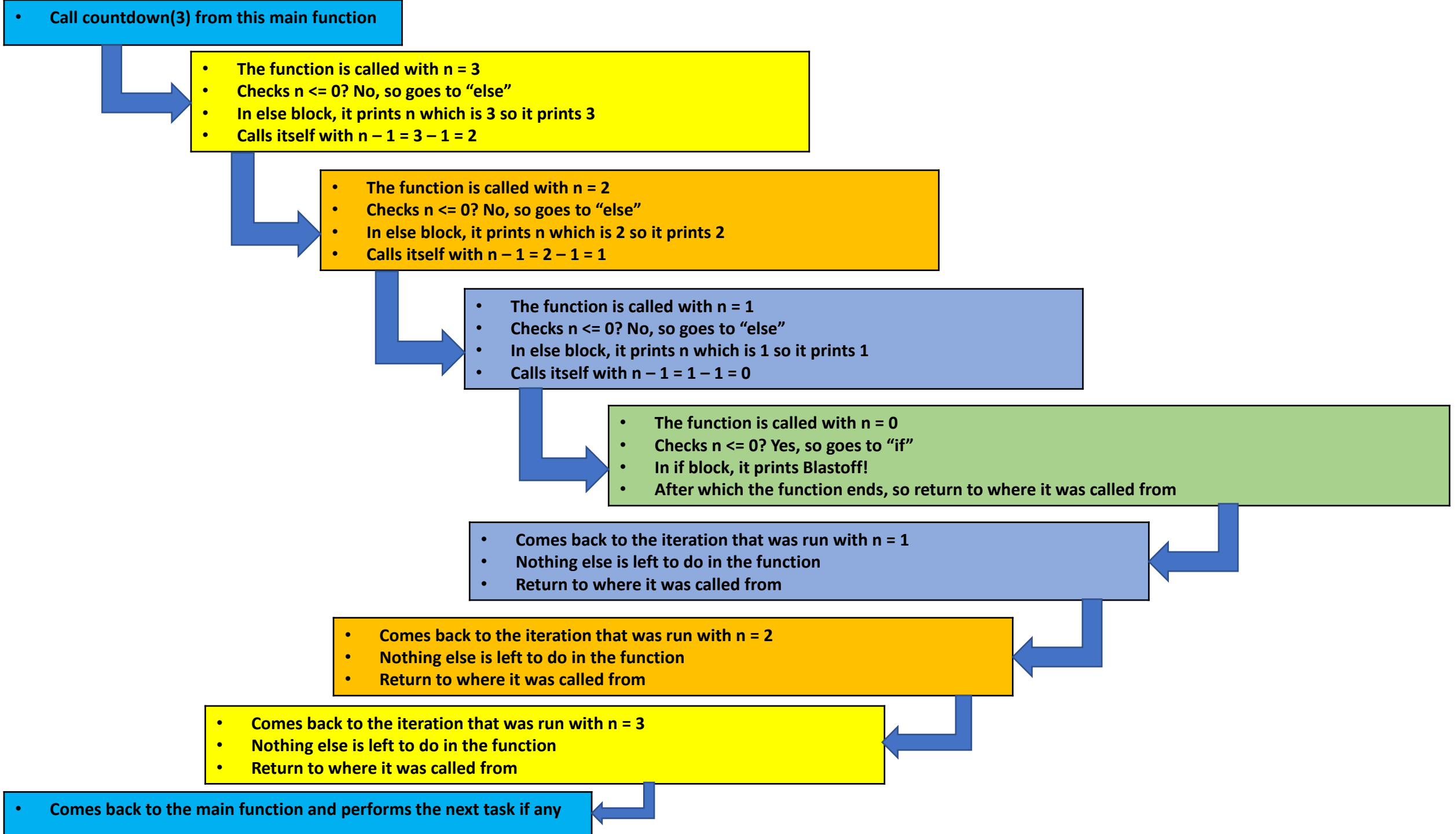
September 30th, 2022

October 5th, 2022

October 7th, 2022

RECURSION

- A process in which a function calls itself directly or indirectly is called ***recursion***
- The function that is repeatedly called is known as ***recursive function***
- Example:
  - def countdown(n):
    if(n <= 0):
    print(“Blastoff!”)
    else:
    print(n)
    countdown(n – 1)
  - In main program, let’s say we call the function countdown with argument 3
    i.e. countdown(3)


Call countdown(3) from this main function
The function is called with n = 3
Checks n <= 0? No, so goes to “else”
In else block, it prints n which is 3 so it prints 3
Calls itself with n – 1 = 3 – 1 = 2
The function is called with n = 2
Checks n <= 0? No, so goes to “else”
In else block, it prints n which is 2 so it prints 2
Calls itself with n – 1 = 2 – 1 = 1
The function is called with n = 1
Checks n <= 0? No, so goes to “else”
In else block, it prints n which is 1 so it prints 1
Calls itself with n – 1 = 1 – 1 = 0
The function is called with n = 0
Checks n <= 0? Yes, so goes to “if”
In if block, it prints Blastoff!
After which the function ends, so return to where it was called from
Comes back to the iteration that was run with n = 1
Nothing else is left to do in the function
Return to where it was called from
Comes back to the iteration that was run with n = 2
Nothing else is left to do in the function
Return to where it was called from
Comes back to the iteration that was run with n = 3
Nothing else is left to do in the function
Return to where it was called from
Comes back to the main function and performs the next task if any

RECURSION

- Output?

RECURSION

- Example:
  - def print_n(s, n):
    ```
    if(n <= 0):
        print("The End!")
    else:
        print(s)
        print_n(s, n – 1)
    ```
  - In main program, let's say we call the function:
    i.e. print_n("Hello", 4)
- Output?

RECURSION

- Example:
  - Determine the sum of first “n” natural numbers

RECURSION

- Example:
  - def add_n(n):
    ```
    if(n <= 1):
        return 1
    else:
        next = add_n(n – 1)
        return (n + next)
        #return(n + add_n(n – 1))
    ```
  - In main program, let's say we call the function:
    ```
    i.e. add_n(10)
    ```
- Output?

RECURSION

- Base Condition:
  - A base case solution added to recursive functions
  - If no base case specified, it won’t know where to stop
  - Usually chosen as the smallest and most obvious condition
  - Recursive functions usually work backwards

STACKS

- A collection of objects that are inserted and removed using the Last-In-First-Out (LIFO) approach
- Can insert elements in stack at any time
- Remove the elements ONLY from top of the stack

FUNCTION FRAMES

- A frame is like a box with the function name besides it and all parameters and/or variables are inside it
- Every time a function is called, it creates a frame of the function and save it on stack, including the main function

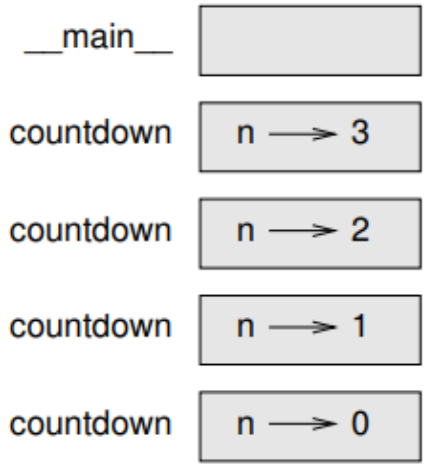


Figure 5.1: Stack diagram.

FUNCTION FRAMES

- Example:
  - Draw a stack diagram for print_n called with s = “Hello” and n = 2

INFINITE RECURSION

- Either no base case present, or base case never reached
- It will continue making recursive calls forever and the program never terminates
- For example:
  - def recurse():
    statement(s)
    …
    recurse()
- Many program environments do not actually run forever to prevent any errors
- Generates an error message after stack is filled to its limit completely

RECURSION EXAMPLE # 1

- Write a function to find Fibonacci series of n numbers, where n > 2,
  i.e. **0, 1, 1, 2, 3, 5, 8, 13, 21, …**

- def fib(n):

```
    #Base conditions
    if (n == 1):
        return 0
    elif (n == 2):
        return 1
    else:
        return (fib(n – 1) + fib(n – 2))
```

RECURSION EXAMPLE # 1

- Main program
  - n = 5
    print(“Fibonacci of ” + n + “ numbers is: ”, end = “ ”)

    for i in range(0, n):
        print(fib(i), end = “ ”)

- Output = ?

RECURSION EXAMPLE # 2

- Write a function that calculates Factorial value of n using recursion, where n > 2

- def fact(n):
```
    #Base conditions
    if (n == 0 or n == 1):
        return 1
    else:
        return (n * fact(n – 1))
```

RECURSION EXAMPLE # 2

- Main program
  - n = 5
    print(“Factorial of ” + n + “ is:”, str(fact(n)))

- Output = ?

RECURSION EXAMPLE # 2

- def fact(n):

```
    #Base conditions
    if (n == 100):
        return 1
    else:
        return (n * fact(n – 1))
```

- Output → ?
- Wrong base case which may cause stack overflow

RECURSION EXAMPLE # 3

- Write a function that calculates $\log_2 n$ using recursion, where n > 1

- ```
  def  logbase2(n):
      #Base condition
      if (n == 1):
          return 0
      else:
          return (1 + logbase2(n//2))
  ```
  ```

RECURSION EXAMPLE # 3

- Main program
  - n = 15
    print(“Log base 2 of ” + n + “ is:”, str(logbase2(n)))

- Output = ?

RECURSION EXAMPLE # 4

- Write a function that prints octal equivalent value of decimal number, n using recursion, where n >= 0

- def dectooct(n):
```
    #Base condition
    if (n == 0):
        print('', end = '')    #empty strings not even blank spaces
    else:
        dectooct(n//8)
        print(n%8, end='')
```

RECURSION EXAMPLE # 4

- Main program
  - n = 45
    dectooct(n)

- Output = ?

RECURSION EXAMPLE # 5

- Write a function that calculates the power of 2 for “n” value using recursion, where n >= 0

-
```
def  powerof2(n):
        #Base condition
        if (n == 0):
                return 1
        else:
                val = 2 * powerof2(n – 1)
                return val
```

RECURSION EXAMPLE # 5

- Main program
  - n = 4
    print(“The result of 2 to the power of ”, n, “is:”, powerof2(n))

- Output = ?

RECURSION EXAMPLE # 6

- Write a recursive function named sum_odd that takes two parameters a and b and returns the sum of all odd numbers between a and b inclusive

-
```
def sum_odd(a,b):
        if (b < a):
                return 0
        else:
                if (b % 2 == 0):
                        return(sum_odd(a,b - 1))
                else:
                        return(sum_odd(a,b-2) + b)
```

RECURSION EXAMPLE # 6

- Main program
  - a = 4, b = 10
    print(sum_odd(a,b))

- Output = ?